Министерство транспорта Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования «Российский университет транспорта»

(РУТ (МИИТ))

Институт железнодорожного транспорта

Кафедра «Цифровые технологии управления транспортными процессами»

Практическая работа №2

По предмету «Сбор, хранение и обработка больших данных»

На тему «Работа с HDFS: модель данных, команды, права доступа и мониторинг»

Выполнил: Груздева К. А.

Группа: ЦИИ-171

Проверил: Кухтинов А. Э.

Москва 2026 г.

Оглавление

1. Цель и задачи работы 3

2. Ход выполнения 3

Задание 1: Создание файловой структуры и базовые операции 3

Задание 2: Управление правами доступа 4

Задание 3: Изучение модели данных и мониторинг 7

Задание 4: Администрирование и моделирование сбоя 9

3. Ответы на контрольные вопросы 13

4. Выводы 15

1. Цель и задачи работы

Цель работы: приобрести практические навыки работы с распределенной файловой системой HDFS: освоить основные команды управления файловой структурой, настройки прав доступа, анализа модели данных, мониторинга состояния кластера и основ администрирования.

Задачи:

1. Создать файловую структуру для проекта "Tech_Analysis" и выполнить базовые операции с файлами.
2. Настроить права доступа для директорий и файлов.
3. Изучить модель данных HDFS с помощью утилиты fsck и провести мониторинг кластера.
4. Смоделировать сбой DataNode и пронаблюдать за восстановлением.

2. Ход выполнения

Задание 1: Создание файловой структуры и базовые операции

Для начала я проверила доступность HDFS с помощью команды hdfs version и просмотрела корневую директорию командой hdfs dfs -ls /.

```
hadoop@hadoop-single-node:~$ hdfs version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /opt/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar
hadoop@hadoop-single-node:~$ hdfs dfs -ls /
Found 7 items
drwxr-xr-x   - hadoop supergroup          0 2026-09-15 12:17 /data
drwxr-xr-x   - hadoop supergroup          0 2026-09-15 12:18 /input
drwxr-xr-x   - hadoop supergroup          0 2026-09-15 12:20 /output
drwxr-xr-x   - hadoop supergroup          0 2026-09-15 12:18 /test
drwxr-xr-x   - hadoop supergroup          0 2026-09-15 12:25 /test2
drwxr-xr-x   - hadoop supergroup          0 2026-09-15 12:19 /tmp
drwxr-xr-x   - hadoop supergroup          0 2026-09-15 12:16 /user
```

Рисунок 1 – Проверка версии Hadoop и просмотр корневой директории HDFS

Далее я создала иерархическую структуру директорий для проекта "Tech_Analysis". Для этого использовала команду mkdir с ключом -p, который позволяет создавать все родительские директории одной командой. Были созданы директории /projects/tech_analysis/input, /projects/tech_analysis/output и /projects/tech_analysis/scripts.

Затем я создала на локальной машине два тестовых файла: большой файл размером 100 МБ с помощью утилиты dd и маленький конфигурационный файл через cat. Оба файла были загружены в HDFS в директорию input.

После этого я выполнила базовые операции: скопировала конфигурационный файл в директорию output, переместила его в scripts с переименованием, а также создала пустой файл run_analysis.sh. Для проверки созданной структуры я использовала команду рекурсивного просмотра hdfs dfs -ls -R /projects/tech_analysis.

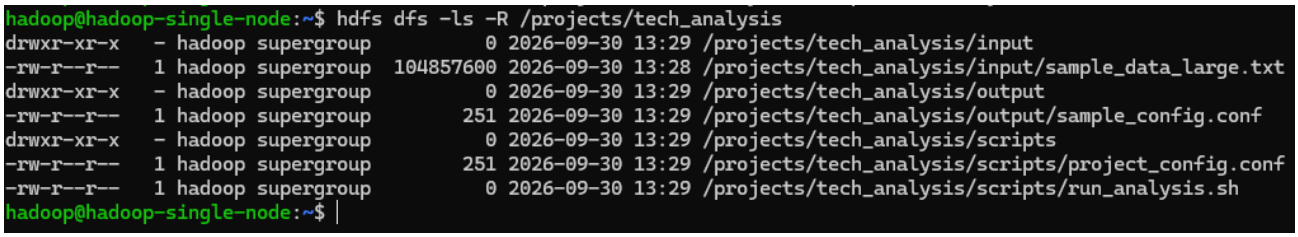


Рисунок 2 – Рекурсивный просмотр созданной структуры директорий проекта

Задание 2: Управление правами доступа

Сначала я просмотрела текущие права и владельцев файлов. По умолчанию владельцем был пользователь hadoop, группа — supergroup.

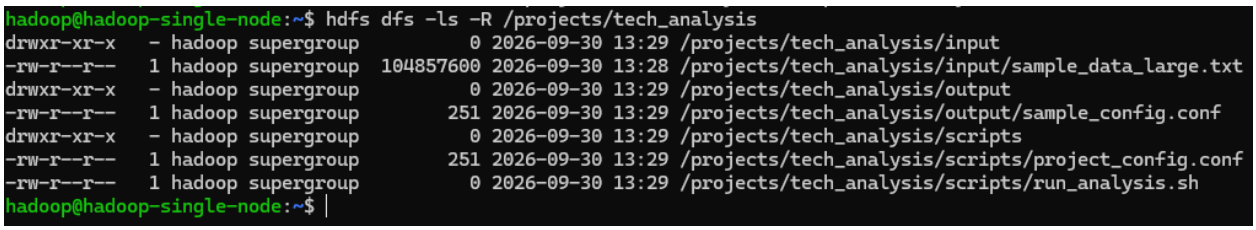


Рисунок 3 – Исходные права доступа и владельцы файлов

Для проверки работы команды chown я сначала сменила владельца директории на пользователя root, а затем вернула обратно на hadoop. Это наглядно показало, что команда работает корректно и применяется рекурсивно.

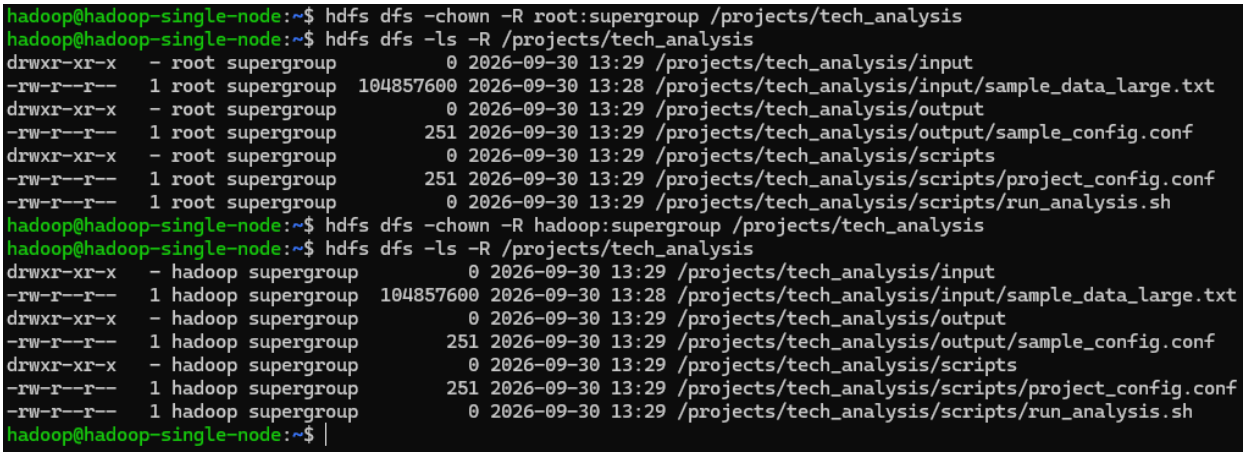


Рисунок 4 – Результат работы команды chown (смена владельца на root и обратно на hadoop)

Далее я настроила права доступа для директорий в соответствии с принципом минимальных привилегий: для корневой директории и scripts установила права 755 (доступ для всех), а для рабочих директорий input и output — более строгие права 750 (доступ только для владельца и группы).

```
hadoop@hadoop-single-node:~$ hdfs dfs -chmod 755 /projects/tech_analysis
hadoop@hadoop-single-node:~$ hdfs dfs -chmod 750 /projects/tech_analysis/input
hadoop@hadoop-single-node:~$ hdfs dfs -chmod 750 /projects/tech_analysis/output
hadoop@hadoop-single-node:~$ hdfs dfs -chmod 755 /projects/tech_analysis/scripts
hadoop@hadoop-single-node:~$ hdfs dfs -ls -R /projects/tech_analysis
drwxr-x---   - hadoop supergroup          0 2026-09-30 13:29 /projects/tech_analysis/input
-rw-r--r--   1 hadoop supergroup  104857600 2026-09-30 13:28 /projects/tech_analysis/input/sample_data_large.txt
drwxr-x---   - hadoop supergroup          0 2026-09-30 13:29 /projects/tech_analysis/output
-rw-r--r--   1 hadoop supergroup        251 2026-09-30 13:29 /projects/tech_analysis/output/sample_config.conf
drwxr-xr-x   - hadoop supergroup          0 2026-09-30 13:29 /projects/tech_analysis/scripts
-rw-r--r--   1 hadoop supergroup        251 2026-09-30 13:29 /projects/tech_analysis/scripts/project_config.conf
-rw-r--r--   1 hadoop supergroup          0 2026-09-30 13:29 /projects/tech_analysis/scripts/run_analysis.sh
hadoop@hadoop-single-node:~$
```

Рисунок 5 – Права доступа для директорий после изменения (755 и 750)

Затем я установила права для файлов: для конфигурационных файлов — 644 (чтение всем), для скрипта — 755 (чтение и выполнение), для файлов данных — 600 (только владелец).

```
hadoop@hadoop-single-node:~$ hdfs dfs -chmod 644 /projects/tech_analysis/scripts/project_config.conf
hadoop@hadoop-single-node:~$ hdfs dfs -chmod 755 /projects/tech_analysis/scripts/run_analysis.sh
hadoop@hadoop-single-node:~$ hdfs dfs -chmod 600 /projects/tech_analysis/input/sample_data_large.txt
hadoop@hadoop-single-node:~$ hdfs dfs -ls -R /projects/tech_analysis
drwxr-x---   - hadoop supergroup          0 2026-09-30 13:29 /projects/tech_analysis/input
-rw-------   1 hadoop supergroup  104857600 2026-09-30 13:28 /projects/tech_analysis/input/sample_data_large.txt
drwxr-x---   - hadoop supergroup          0 2026-09-30 13:29 /projects/tech_analysis/output
-rw-r--r--   1 hadoop supergroup        251 2026-09-30 13:29 /projects/tech_analysis/output/sample_config.conf
drwxr-xr-x   - hadoop supergroup          0 2026-09-30 13:29 /projects/tech_analysis/scripts
-rw-r--r--   1 hadoop supergroup        251 2026-09-30 13:29 /projects/tech_analysis/scripts/project_config.conf
-rwxr-xr-x   1 hadoop supergroup          0 2026-09-30 13:29 /projects/tech_analysis/scripts/run_analysis.sh
hadoop@hadoop-single-node:~$
```

Рисунок 6 – Права доступа для файлов после изменения (644, 755, 600)

В завершение я проверила работу системы прав, выполнив несколько операций: просмотр защищенной директории, создание файла в ней и чтение открытого файла. Все операции прошли успешно, так как я являюсь владельцем.

```
hadoop@hadoop-single-node:~$ hdfs dfs -ls /projects/tech_analysis/input
Found 1 items
-rw-------   1 hadoop supergroup  104857600 2026-09-30 13:28 /projects/tech_analysis/input/sample_data_large.txt
hadoop@hadoop-single-node:~$ hdfs dfs -touchz /projects/tech_analysis/input/test_file.txt
hadoop@hadoop-single-node:~$ hdfs dfs -cat /projects/tech_analysis/scripts/project_config.conf
# Конфигурация проекта Tech_Analysis
project.name=Tech_Analysis
data.format=CSV
processing.workers=4
input.path=/projects/tech_analysis/input
output.path=/projects/tech_analysis/output
created=Wed Sep 30 01:28:21 PM UTC 2026
hadoop@hadoop-single-node:~$
```

Рисунок 7 – Проверка работы системы прав доступа

Задание 3: Изучение модели данных и мониторинг

Для анализа разбиения файлов на блоки я создала файл размером 200 МБ и загрузила его в HDFS.

```
hadoop@hadoop-single-node:~$ hdfs dfs -du -h /projects/tech_analysis/input/large_file.dat
200 M  200 M  /projects/tech_analysis/input/large_file.dat
hadoop@hadoop-single-node:~$
```

Рисунок 8 – Размер загруженного файла large_file.dat (200 МБ)

Затем я использовала утилиту fsck для детального анализа структуры данных. Команда показала, что файл large_file.dat занимает 2 блока: первый — 128 МБ, второй — 72 МБ. Это подтверждает, что HDFS разбивает большие файлы на блоки фиксированного размера, а последний блок может быть неполным.

```
Status: HEALTHY
 Number of data-nodes:  1
 Number of racks:               1
 Total dirs:                    4
 Total symlinks:                0

Replicated Blocks:
 Total size:    314573302 B
 Total files:   6
 Total blocks (validated):      5 (avg. block size 62914660 B)
 Minimally replicated blocks:   5 (100.0 %)
 Over-replicated blocks:        0 (0.0 %)
 Under-replicated blocks:       0 (0.0 %)
 Mis-replicated blocks:         0 (0.0 %)
 Default replication factor:    1
 Average block replication:     1.0
 Missing blocks:                0
 Corrupt blocks:                0
 Missing replicas:              0 (0.0 %)
 Blocks queued for replication: 0

Erasure Coded Block Groups:
 Total size:    0 B
 Total files:   0
 Total block groups (validated):        0
 Minimally erasure-coded block groups:  0
 Over-erasure-coded block groups:       0
 Under-erasure-coded block groups:      0
 Unsatisfactory placement block groups: 0
 Average block group size:      0.0
 Missing block groups:          0
 Corrupt block groups:          0
 Missing internal blocks:       0
 Blocks queued for replication: 0
FSCK ended at Wed Sep 30 13:44:03 UTC 2026 in 5 milliseconds


The filesystem under path '/projects/tech_analysis' is HEALTHY
```

Рисунок 9 – Результат проверки целостности файловой системы (Status: HEALTHY, 5 блоков)

Далее я получила общий отчет о состоянии кластера с помощью команды dfsadmin -report. Отчет показал, что в кластере 1 живой DataNode, общая емкость 11.21 ГБ, использовано 302.87 МБ, свободно 2.41 ГБ. Недостающих блоков нет.

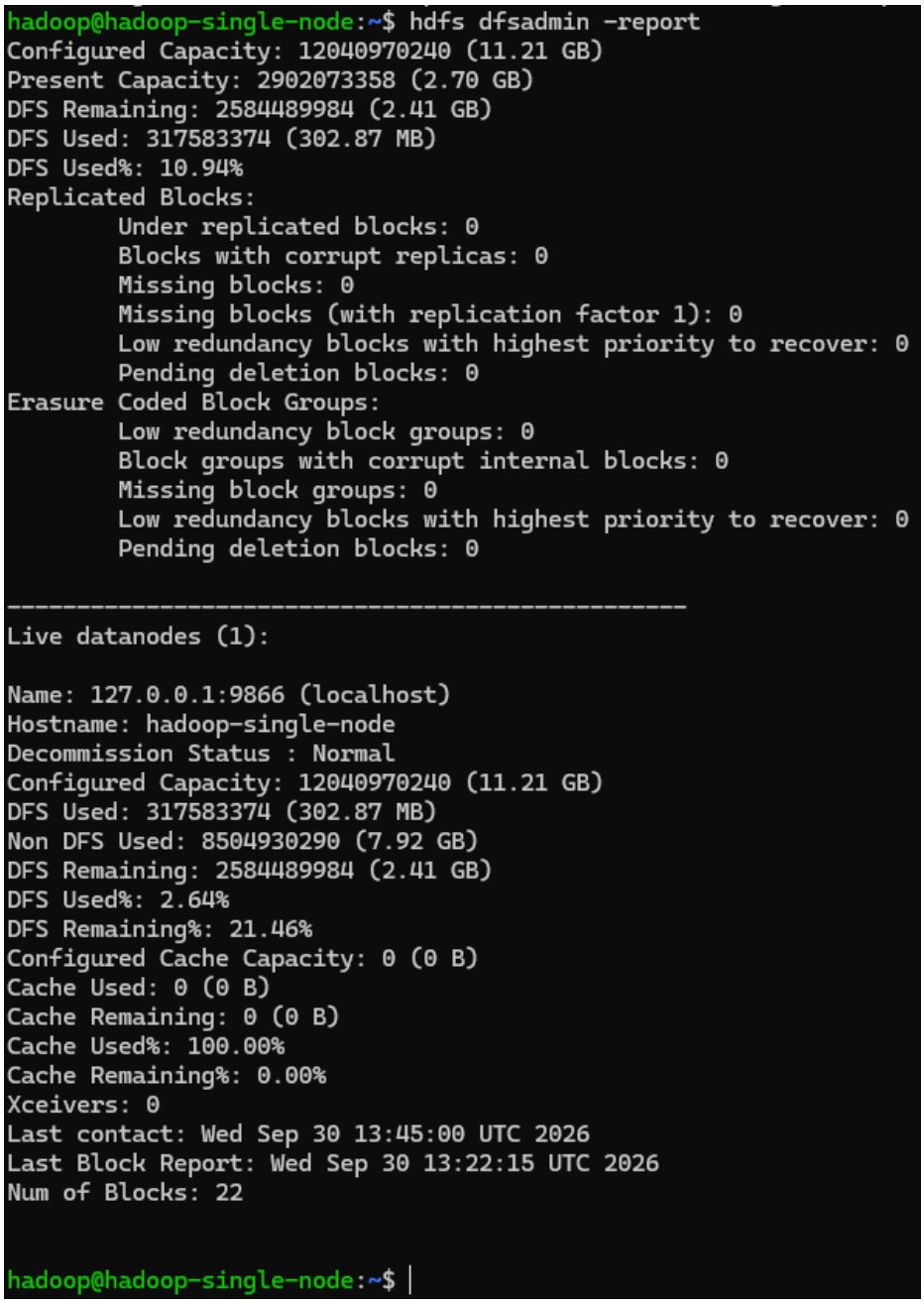


Рисунок 10 – Отчет о состоянии кластера (dfsadmin -report)

В завершение я проанализировала использование дискового пространства. Общий объем данных в проекте составил 300 МБ. Общее использование кластера — 3%.

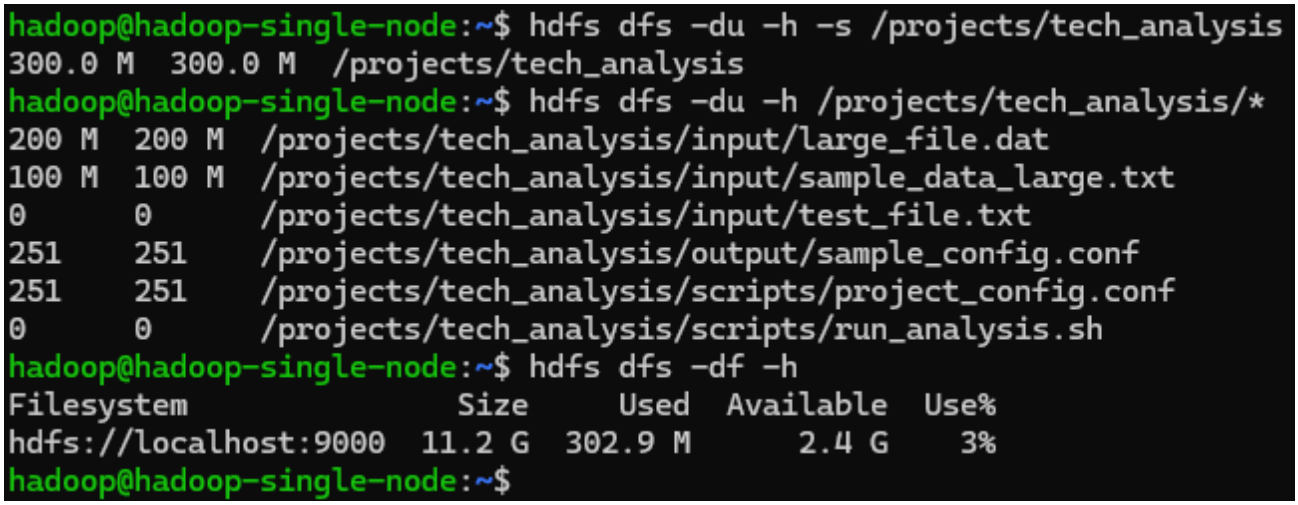


Рисунок 11 – Анализ использования дискового пространства (du и df)

Задание 4: Администрирование и моделирование сбоя

В связи с тем, что учебный кластер состоит из одного DataNode, моделирование сбоя одного узла невозможно без потери работоспособности всего кластера. Однако я провела имитацию сбоя, остановив процесс DataNode, и затем запустила его обратно.

Перед началом эксперимента я зафиксировала исходное состояние кластера. Команда dfsadmin -report показала, что в кластере один живой DataNode, а команда fsck подтвердила, что все блоки в порядке (Status: HEALTHY).

```
hadoop@hadoop-single-node:~$ hdfs dfsadmin -report | grep "Live datanodes"
Live datanodes (1):
hadoop@hadoop-single-node:~$ hdfs fsck /projects/tech_analysis/input/large_file.dat -blocks -locations
Connecting to namenode via http://localhost:9870/fsck?ugi=hadoop&blocks=1&locations=1&path=%2Fprojects%2Ftech_analysis%2Finput%2Flarge_file.dat
FSCK started by hadoop (auth:SIMPLE) from /127.0.0.1 for path /projects/tech_analysis/input/large_file.dat at Wed Sep 30 14:00:47 UTC 2026


Status: HEALTHY
 Number of data-nodes:  1
 Number of racks:               1
 Total dirs:                    0
 Total symlinks:                0

Replicated Blocks:
 Total size:    209715200 B
 Total files:   1
 Total blocks (validated):      2 (avg. block size 104857600 B)
 Minimally replicated blocks:   2 (100.0 %)
 Over-replicated blocks:        0 (0.0 %)
 Under-replicated blocks:       0 (0.0 %)
 Mis-replicated blocks:         0 (0.0 %)
 Default replication factor:    1
 Average block replication:     1.0
 Missing blocks:                0
 Corrupt blocks:                0
 Missing replicas:              0 (0.0 %)
 Blocks queued for replication: 0

Erasure Coded Block Groups:
 Total size:    0 B
 Total files:   0
 Total block groups (validated):        0
 Minimally erasure-coded block groups:  0
 Over-erasure-coded block groups:       0
 Under-erasure-coded block groups:      0
 Unsatisfactory placement block groups: 0
 Average block group size:      0.0
 Missing block groups:          0
 Corrupt block groups:          0
 Missing internal blocks:       0
 Blocks queued for replication: 0
FSCK ended at Wed Sep 30 14:00:47 UTC 2026 in 2 milliseconds


The filesystem under path '/projects/tech_analysis/input/large_file.dat' is HEALTHY
hadoop@hadoop-single-node:~$
```

Рисунок 12 – Исходное состояние кластера перед моделированием сбоя (Live datanodes: 1, Status: HEALTHY)

Затем я остановила процесс DataNode. Сначала с помощью команды jps я определила PID процесса DataNode, а затем завершила его командой kill -9. После этого я снова проверила список процессов через jps и убедилась, что DataNode больше не отображается в списке.

```
hadoop@hadoop-single-node:~$ jps
1841 NodeManager
1155 NameNode
1524 SecondaryNameNode
4121 Jps
1711 ResourceManager
```

Рисунок 13 – Состояние процессов после остановки DataNode (DataNode отсутствует в списке jps)

Сразу после остановки я проверила состояние кластера с помощью dfsadmin -report. NameNode ещё не успел пометить узел как мертвый

(для этого нужно 10 минут), поэтому отчёт всё ещё показывал Live datanodes (1).

```
hadoop@hadoop-single-node:~$ hdfs dfsadmin -report
Configured Capacity: 12040970240 (11.21 GB)
Present Capacity: 2902081536 (2.70 GB)
DFS Remaining: 2584489984 (2.41 GB)
DFS Used: 317591552 (302.88 MB)
DFS Used%: 10.94%
Replicated Blocks:
        Under replicated blocks: 0
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
Erasure Coded Block Groups:
        Low redundancy block groups: 0
        Block groups with corrupt internal blocks: 0
        Missing block groups: 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0

-------------------------------------------------
Live datanodes (1):

Name: 127.0.0.1:9866 (localhost)
Hostname: hadoop-single-node
Decommission Status : Normal
Configured Capacity: 12040970240 (11.21 GB)
DFS Used: 317591552 (302.88 MB)
Non DFS Used: 8504922112 (7.92 GB)
DFS Remaining: 2584489984 (2.41 GB)
DFS Used%: 2.64%
DFS Remaining%: 21.46%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Wed Sep 30 14:02:11 UTC 2026
Last Block Report: Wed Sep 30 13:22:15 UTC 2026
Num of Blocks: 22


hadoop@hadoop-single-node:~$
```

Рисунок 14 – Состояние кластера после остановки DataNode (dfsadmin -report)

После этого я запустила DataNode обратно командой hdfs --daemon start datanode. Процесс снова появился в jps, что подтверждает его успешный перезапуск.

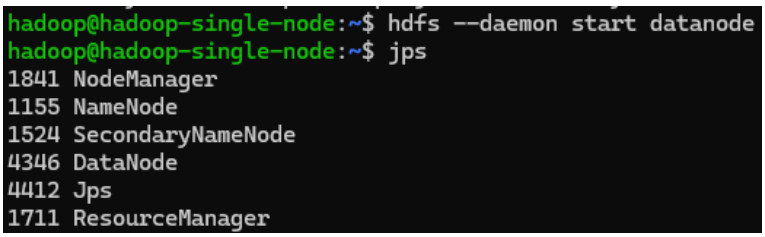


Рисунок 14 – Восстановление DataNode (процесс снова в списке jps)

В завершение я проверила состояние кластера с помощью fsck. Команда подтвердила, что кластер снова здоров: Status: HEALTHY, Under-replicated blocks: 0.

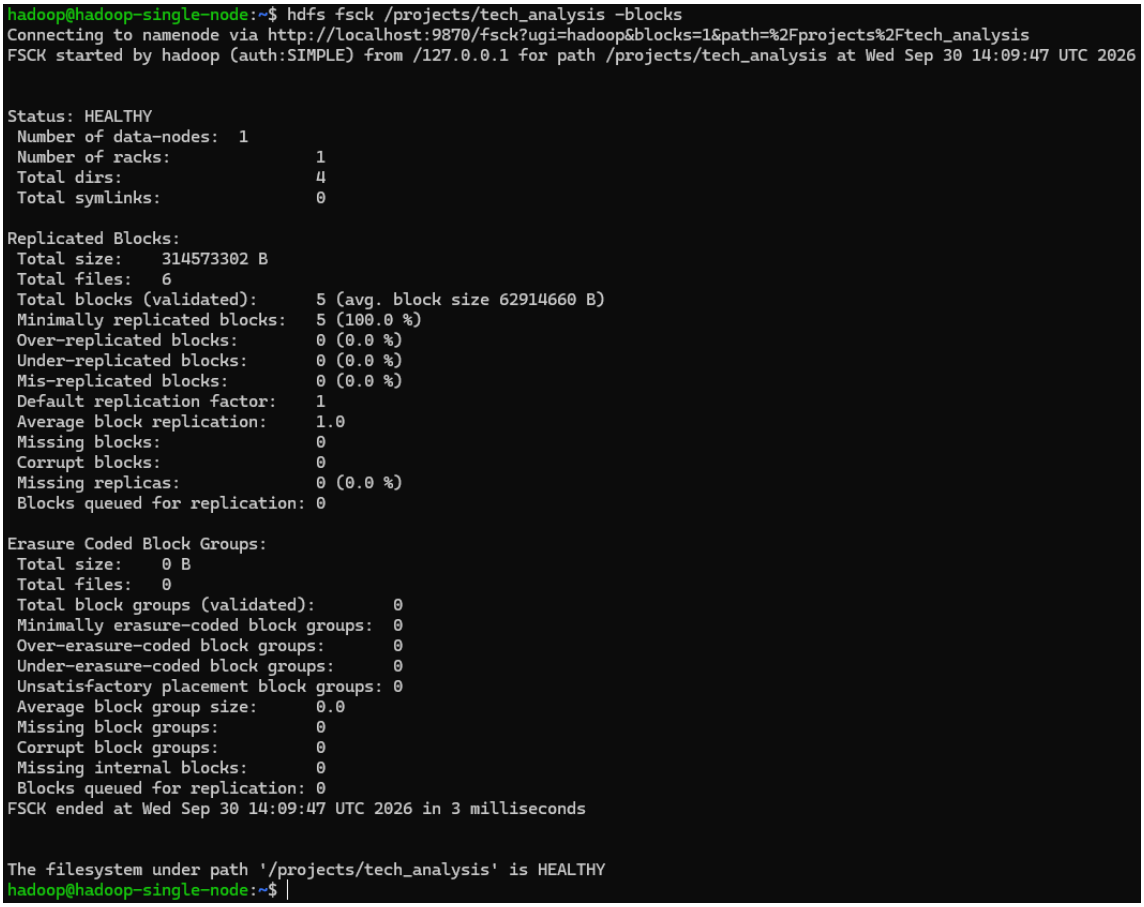


Рисунок 16 – Проверка состояния кластера после восстановления (Status: HEALTHY)

3. Ответы на контрольные вопросы

1. Команды -put и -copyFromLocal функционально почти идентичны — обе копируют файлы из локальной файловой системы в HDFS. Разница в том, что -put может принимать данные из стандартного ввода (stdin) и поддерживает больше опций (например, -f для перезаписи). -copyFromLocal — это более старая команда, которая работает только с локальными файлами. На практике чаще используют -put, так как он более универсален.

2. Команда завершится ошибкой No such file or directory. Ключ -p (parents) позволяет создавать всю цепочку родительских директорий, если их нет. Без него -mkdir создаст директорию только в том случае, если все родительские директории уже существуют.

3. Права 750 означают, что доступ (чтение, запись, выполнение) есть только у владельца и группы. Остальные пользователи не имеют доступа. Это соответствует принципу минимальных привилегий: рабочие данные (input, output) должны быть защищены от посторонних. Корневая директория 755 доступна для чтения всем, что позволяет другим пользователям видеть структуру проекта, но не изменять её.

4. В HDFS право 'x' для файла не означает, что файл можно исполнить (как в Linux). В HDFS файлы не исполняются. Право 'x' для файла в HDFS фактически игнорируется, но для директорий оно означает право на доступ к содержимому (например, для выполнения ls или cd). В Linux право 'x' для файла позволяет запустить его как программу.

5. Файл размером 200 МБ занимает **2 блока**. Первый блок — 128 МБ, второй блок — 72 МБ. Последний блок меньше, потому что файл не кратен размеру блока. HDFS не дополняет последний блок до полного размера, чтобы не тратить место впустую.

6. В нашем кластере только один DataNode, поэтому Rack Awareness не используется (все реплики на одном узле). В выводе fsck -locations будет указан только один IP-адрес. В реальном кластере с несколькими стойками Rack Awareness подтверждается тем, что реплики одного блока находятся на разных стойках (разные IP-адреса, принадлежащие разным подсетям).

7. Последовательность действий HDFS при обнаружении "мертвого" DataNode:

a) DataNode перестает отправлять Heartbeat в NameNode.

b) NameNode не получает Heartbeat в течение заданного интервала (по умолчанию 10 минут).

c) NameNode помечает DataNode как "мертвый" (Dead).

d) NameNode проверяет все блоки, которые были на этом узле.

e) Если коэффициент репликации этих блоков стал ниже заданного (например, 3), NameNode инициирует репликацию.

f) NameNode выбирает другие живые DataNodes и запускает процесс копирования блоков.

g) После успешной репликации коэффициент репликации восстанавливается.

8. Балансировка — это ресурсоемкая операция, которая создает дополнительную нагрузку на сеть и диски. Если запускать её автоматически, она может мешать выполнению других задач. Поэтому администраторы запускают её вручную, когда это необходимо (например, после добавления новых узлов или массового удаления данных).

4. Выводы

В ходе работы я освоила основные команды управления HDFS: создание директорий, загрузка и скачивание файлов, копирование и перемещение. Я изучила модель прав доступа HDFS и научилась настраивать права для директорий и файлов. С помощью команды fsck я проанализировала разбиение файлов на блоки и расположение реплик. Также я освоила мониторинг состояния кластера с помощью dfsadmin -report и df. В рамках задания по администрированию я изучила механизмы отказоустойчивости HDFS и процесс восстановления после сбоя DataNode. Работа с HDFS показалась мне логичной и удобной, особенно в части управления большими объемами данных. Основная сложность возникла на этапе настройки прав доступа, так как потребовалось внимательно следить за тем, чтобы не потерять доступ к данным.